

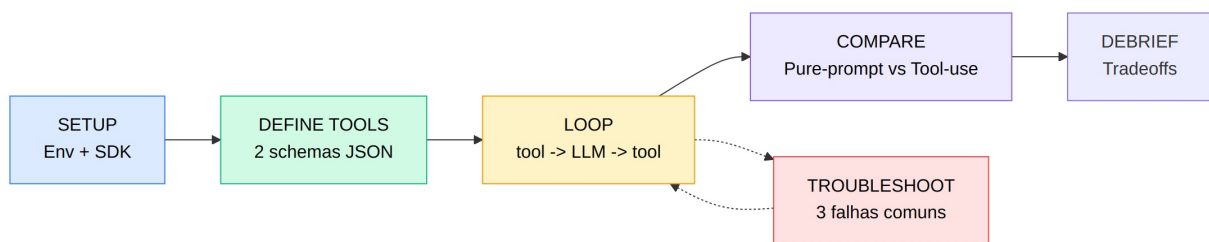
Lab 1: Agent CLI Single-Turn com Tool-use

Tipo: Projeto | **Duração:** 90 min | **Nível:** Intermediário | **Bloom:** Apply

Objetivos de Aprendizagem

O lab cobre dois objetivos do módulo. O primeiro (M1-O4) é implementar function-calling, ou seja, declarar tools via schema JSON e construir um loop de execução que orquestra LLM e ferramentas em um agente single-turn. O segundo (M1-O5) é comparar abordagens de pure-prompt e tool-use sobre o mesmo problema, identificando em quais situações cada uma é apropriada.

Fluxo do Lab



Fluxo do Lab 1: setup, definição de tools, loop tool-LLM-tool, comparativo e debrief

O caminho percorre cinco etapas. Começa pelo setup do provider, passa pela definição de duas tools com schema JSON, exercita o loop em que o LLM decide chamar uma tool e depois consome o resultado, faz um comparativo direto entre pure-prompt e tool-use e fecha com um debrief sobre tradeoffs. O bloco de troubleshooting aparece ao lado porque é consultado de forma iterativa enquanto o loop é depurado.

Pré-requisitos

Requisito	Versão
Python	3.10+
openai (SDK)	latest
pydantic	2.x
python-dotenv	1.x

Default da turma: **Gemini Free Tier**. Setup completo de provider, quota e troubleshooting em [notebooks/API-KEYS.pdf](#).

Setup (escolha um provider)

Inicializar o projeto:

```
uv venv && source .venv/bin/activate
uv pip install openai pydantic python-dotenv
```

Usamos o SDK `openai` para os três providers, porque todos expõem endpoint OpenAI-compatible. O que muda é apenas a `base_url` e o nome do modelo.

Opção A. Gemini Free Tier (default):

```
echo 'GEMINI_API_KEY=AIza...' >> .env
```

```

import json
import os
from typing import Any
from openai import OpenAI
from pydantic import BaseModel, Field

client = OpenAI(
    api_key=os.environ["GEMINI_API_KEY"],
    base_url="https://generativelanguage.googleapis.com/v1beta/openai/",
)
MODEL = "gemini-2.5-flash-lite"  # 15 RPM, 1000 RPD; suba para gemini-2.5-flash se tool-use ficar instável

```

Opção B. OpenAI (\$5 free credit):

```

client = OpenAI()  # OPENAI_API_KEY do .env
MODEL = "gpt-4o-mini"

```

Opção C. Ollama 100% local (Qwen3 é o mais estável em tool-use):

```
ollama pull qwen3:8b && ollama serve
```

```

client = OpenAI(
    api_key="ollama",  # qualquer string
    base_url="http://localhost:11434/v1",
)
MODEL = "qwen3:8b"

```

Atenção ao comportamento de tool-use no Gemini Flash-Lite. Quando a query é ambígua, o modelo às vezes decide não chamar a tool. Se isso acontecer durante o lab, suba para `gemini-2.5-flash` ou force `tool_choice` (veja Troubleshooting, problema 1).

Etapa 1. Definir duas tools com schema JSON

A primeira tool é uma calculadora que faz `eval` seguro de expressões aritméticas, e a segunda é uma busca em corpus local de documentação.

```

def calculator(expression: str) -> str:
    """Avalia expressao aritmetica simples."""
    allowed = set("0123456789+-*/(). ")
    if not all(c in allowed for c in expression):
        return "ERROR: expressao contem caracteres nao permitidos"
    try:
        return str(eval(expression, {"__builtins__": {}}, {}))
    except Exception as e:
        return f"ERROR: {e}"

DOCS = {
    "retry": "Use exponential backoff entre tentativas para evitar thundering herd.",
    "pydantic": "Pydantic valida payload via BaseModel + type hints.",
    "streaming": "Streaming reduz time-to-first-token mas dificulta retry idempotente.",
}

def lookup_doc(term: str) -> str:
    """Consulta documentacao local por termo exato (case-insensitive)."""
    return DOCS.get(term.lower(), f"NOT_FOUND: termo '{term}' nao encontrado")

TOOLS = [
    {
        "type": "function",
        "function": {
            "name": "calculator",
            "description": "Avalia uma expressao aritmetica simples (apenas + - * / e parenteses).",
            "parameters": {
                "type": "object",
                "properties": {
                    "expression": {
                        "type": "string",
                        "description": "Expressao matematica, ex: '12 * (3 + 4)'",
                    }
                },
                "required": ["expression"],
            },
        },
    },
    {
        "type": "function",
        "function": {
            "name": "lookup_doc",
            "description": "Consulta um termo na base de documentacao local (retry, pydantic, streaming).",
            "parameters": {
                "type": "object",
                "properties": {
                    "term": {
                        "type": "string",
                        "description": "Termo a buscar, ex: 'retry'",
                    }
                },
                "required": ["term"],
            },
        },
    },
]

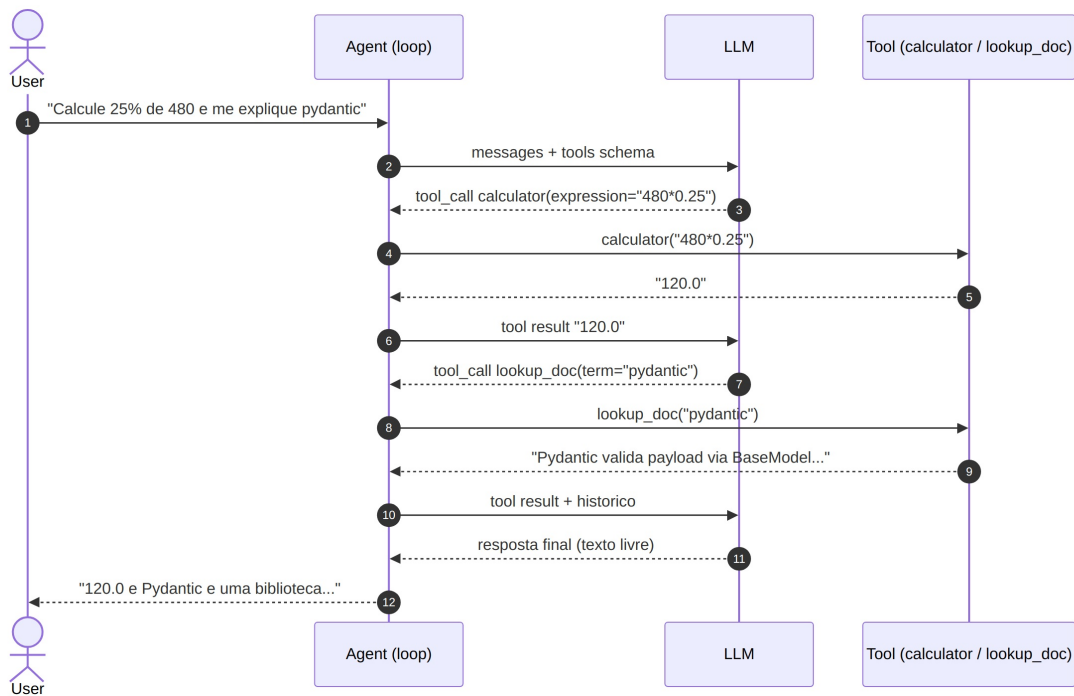
TOOL_REGISTRY = {"calculator": calculator, "lookup_doc": lookup_doc}

```

Pergunta para discussão: por que o `calculator` valida o input antes de chamar `eval`? Que tipo de ataque essa validação previne?

Etapa 2. Loop tool, LLM, tool (Single-Turn Agent)

A figura a seguir mostra como uma única query do usuário pode disparar várias chamadas de tool antes da resposta final.



Sequence diagram do agente: user dispara query, agente loop entre LLM e tools antes de responder

A leitura é direta. O usuário faz uma pergunta. O agente repassa ao LLM junto do schema de tools. O LLM responde com uma `tool_call`. O agente executa a função local e devolve o resultado ao LLM. Quando o LLM já tem o que precisa, ele para de chamar tools e devolve a resposta em texto livre.

```

def run_agent(user_query: str, max_iters: int = 5) -> str:
    messages = [
        {
            "role": "system",
            "content": (
                "Voce e um assistente que pode usar tools. "
                "Use 'calculator' para contas e 'lookup_doc' para definicoes tecnicas. "
                "Sempre cite a fonte quando usar lookup_doc."
            ),
        },
        {"role": "user", "content": user_query},
    ]

    for _ in range(max_iters):
        response = client.chat.completions.create(
            model=MODEL,
            messages=messages,
            tools=TOOLS,
            tool_choice="auto",
        )
        msg = response.choices[0].message
        messages.append(msg.model_dump(exclude_unset=True))

        if not msg.tool_calls:
            return msg.content or ""

        for call in msg.tool_calls:
            fn_name = call.function.name
            args = json.loads(call.function.arguments)
            result = TOOL_REGISTRY[fn_name](**args)
            messages.append(
                {
                    "role": "tool",
                    "tool_call_id": call.id,
                    "content": result,
                }
            )

    return "[ERROR] max iterations excedidas"

print(run_agent("Quanto e 47 * 13 + 200?"))
print(run_agent("O que e retry em chamadas HTTP?"))
print(run_agent("Calcule 25% de 480 e me explique o que e pydantic"))

```

Ponto de verificação: as três queries devem produzir respostas corretas, ou seja, `47*13+200 = 811`, a definição de retry e a combinação de `120` com a definição de pydantic. A terceira query é especialmente importante porque exige duas tool calls consecutivas na mesma conversa.

Etapa 3. Comparativo entre pure-prompt e tool-use

Repita as mesmas três queries sem usar tools, apenas com prompt direto:

```
def run_pure_prompt(user_query: str) -> str:
    response = client.chat.completions.create(
        model=MODEL,
        messages=[
            {"role": "system", "content": "Responda diretamente. Faça contas mentalmente."},
            {"role": "user", "content": user_query},
        ],
    )
    return response.choices[0].message.content or ""

print(run_pure_prompt("Quanto é 47 * 13 + 200?"))
print(run_pure_prompt("O que é retry em chamadas HTTP?"))
print(run_pure_prompt("Calcule 25% de 480 e me explique o que é pydantic"))
```

Pergunta para discussão: compare as seis respostas (três com tool, três sem). Em qual delas o LLM puro errou ou alucinou? Em qual o tool-use foi overkill?

Verificação

Checklist de conclusão do lab:

- Os schemas JSON estão definidos para `calculator` e `lookup_doc` com `parameters.required` populado.
- O `TOOL_REGISTRY` mapeia nomes para funções Python executáveis.
- O loop tool, LLM, tool termina por resposta final ou por `max_iters`.
- As três queries de teste produzem resultados corretos via tool-use.
- O comparativo entre pure-prompt e tool-use está documentado em texto livre (no mínimo três frases).

Tabela comparativa esperada

Query	Pure-prompt	Tool-use	Recomendado
Aritmética com mais de 2 dígitos	erro silencioso ocasional	sempre exato	Tool-use
Definição factual	depende do training cutoff	ancorado em fonte	Tool-use
Conversa aberta	natural	overhead desnecessário	Pure-prompt

Troubleshooting

Problema 1. `tool_calls` é `None` mesmo com `tool_choice="auto"`

O sintoma típico é o LLM responder direto sem usar a tool, mesmo em uma conta complexa. Em geral o system prompt está fraco demais e o LLM não percebe que tem permissão para chamar tool. Reforce a instrução com algo como `"Sempre use 'calculator' para QUALQUER calculo aritmetico."` Outra alternativa é forçar via `tool_choice={"type": "function", "function": {"name": "calculator"}}`.

Problema 2. `json.JSONDecodeError` em `call.function.arguments`

O sintoma é erro ao parsear os argumentos da tool call. Isso costuma indicar que o LLM gerou JSON inválido (raro com modelos novos, comum com modelos antigos). A correção é envolver o `json.loads` em `try/except` e devolver o erro como tool result, por exemplo `"ERROR: invalid arguments"`. O LLM tipicamente faz auto-correção no próximo turno.

Problema 3. Loop infinito (LLM chama a mesma tool repetidamente)

Quando o `max_iters` é excedido sem resposta final, em geral uma tool retornou erro mas o LLM não interpretou e simplesmente repetiu a chamada. Padronize os resultados começando com `OK:` ou `ERROR:` e oriente o system: `"Se uma tool retornar ERROR, NAO repita a mesma chamada; resuma o erro ao usuario."`

Referências

- [OpenAI Function Calling](#)
- [Anthropic Tool Use](#)
- [Pydantic v2 Docs](#)